

第三章

1. (1) C (2) C (3) D (4) A (5) A (6) A (7) A (8) B
(9) B (10) D (11) A (12) D (13) B (14) C (15) B

2. (1) 双栈初始化

```
void initDblStack (DblStack *S, int m)
```

```
    S → top[0] = -1;
```

```
    S → top[1] = m;
```

```
    S → bot[0] = 0;
```

```
    S → bot[1] = m-1;
```

```
    S → V = ElemType* malloc(m);
```

```
    S → m = m;
```

```
}
```

判断栈空

```
Status isClear (DblStack S, int stackNum) {
```

```
    if (stackNum == 0) {
```

```
        return S.top[0] == -1;
```

```
    } else if (stackNum == 1) {
```

```
        return S.top[1] == m;
```

```
} else {
```

```
    return Error;
```

```
}
```

```
}
```

判断栈满

```
Status isFull (DblStack S) {
```

```
    return S.top[1] == S.top[0] + 1;
```

```
}
```

进栈

```
Status pushStack (DblStack* S, int stackNum,  
                  ElemType e) {
```

```
    if (isFull(S)) {
```

```
        return Error;
```

```
}
```

```
    if (stackNum == 0) {
```

```
        S->V[++S->top[0]] = e;
```

```
    } else if (stackNum == 1) {
```

```
        S->V[--S->top[1]] = e;
```

```
} else {
```

```
    return Error;
```

```
}
```

```
return OK;
```

```
}
```

```
Status pop Stack (DblStack* S, int stackNum,  
                  ElemType* e)
```

```
{ if (isclear(S, stackNum))  
    return Error;
```

```
}
```

```
if (stackNum == 0)
```

```
    *e = S->V[S->top[0]--];
```

```
} else if (stackNum == 1)
```

```
    *e = S->V[S->top[1]++];
```

```
} else {
```

```
    return Error;
```

```
}
```

return OK

✓
(b) typedef struct QNode {
 ElemType data;
 struct QNode *next;
} QNode, *QPtr;
typedef struct {
 QPtr rear;
} QLink;

Status ClearQ (QLink * Q)

while (Q->rear != Q->rear->next) {

 Q = Q->rear->next;

 Q->rear->next = Q->next;

 delete S;

 }
 return OK

✓

```
Status isClear (QLink Q){
```

```
    return Q → rear == Q → rear → next;
```

```
}
```

```
Status enqueue (QLink &Q, ElemType e){
```

```
    q = new QNode;
```

```
    q → data = e;
```

```
    p = Q → rear → next;
```

```
    Q → rear → next = q;
```

```
    q → next = p;
```

```
    Q → rear = q;
```

```
    return OK;
```

```
}
```

```
Status dequeue (QLink &Q, ElemType &e){
```

```
    if (isClear){
```

```
        return Error;
```

```
    }
```

```
    p = Q → rear → next → next;
```

```
    e = p → data;
```

```
if (p == Q->next)?
```

```
Q->rear = Q->rear->next;
```

```
Q->rear->next = p->next;
```

```
} else {
```

```
Q->rear->next->next = p->next;
```

```
}
```

```
delete p;
```

```
return OK
```

```
}
```

(7) typedef ET ElemType.

```
status enqueue (ET Q[], int &front, int &rear,
```

```
ET e, bool tag, int m){
```

```
if (front == rear && tag == 1){
```

```
return Error;
```

```
}
```

```
Q[rear] = e;
```

```
rear = (rear + 1) % m;
```

```
    tag = 1 ;  
    return OK
```

```
}
```

```
Status dequeue (ET Q[], int &front, int &rear,  
                ET &e, bool tag, int m)?
```

```
if (front == rear && tag == 0)?
```

```
    return Error;
```

```
}
```

```
e = Q[front];
```

```
front = (front + 1) % m;
```

```
if (front == rear){
```

```
    tag = 0;
```

```
}
```

```
return OK
```

```
}
```

(18) typedef struct {

ElemType data[MaxSize];

int front;

int rear;

} CQ;

Status dequeue_r(CQ* q){

q->rear = (q->rear - 1 + MaxSize) % MaxSize;

return OK;

}

Status enqueue_f(CQ* q, ET e){

q->front = (q->front + 1 + MaxSize) % MaxSize;

q->data[q->front] = e;

return OK;

}